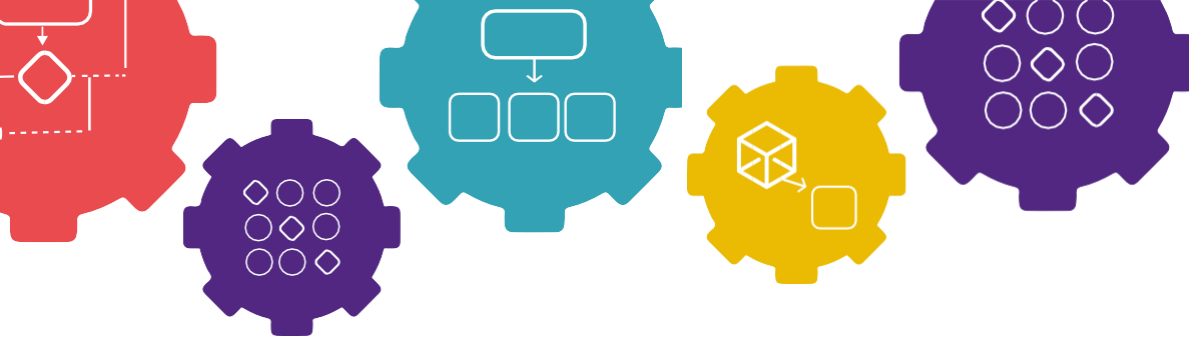




Laboratorio 15



Control de Calidad de las soluciones

Objetivos

- Anticipar y gestionar errores comunes durante el desarrollo de soluciones.
- Emplear técnicas básicas de depuración.



Conceptos clave

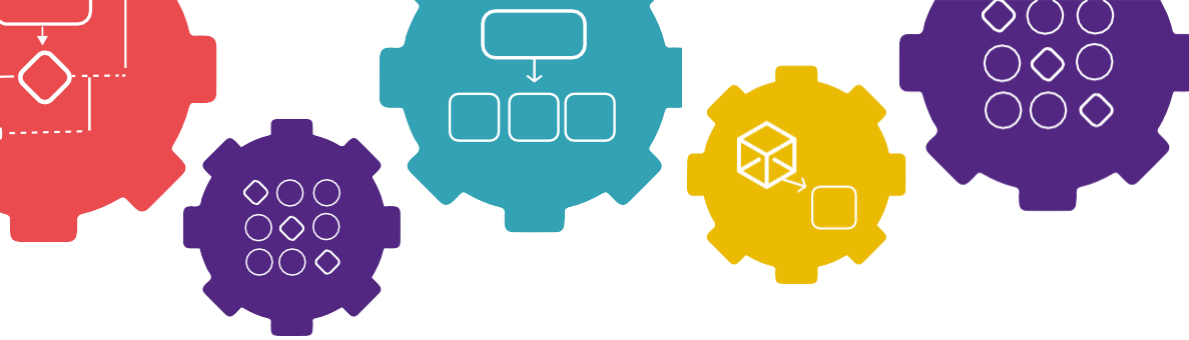
Defecto:

Es una falla en la solución que ocasiona un comportamiento equivocado. Existen varios tipos de defectos:

- De escritura
- Mala gramática y ambigüedad
- Inconsistencias
- Defectos lógicos y matemáticos.

Los defectos pueden ser detectados y eliminados en cualquier etapa del proceso de desarrollo, pero cuanto antes se encuentren será mejor. Algunos defectos aparecerán solo cuando la solución se ha implementado. En estos casos se requieren de pruebas (testing) y seguimiento paso a paso (debugging).

Pruebas de calidad: Verifican que el sistema desarrollado se comporta como se espera e identifica áreas específicas de fallo.

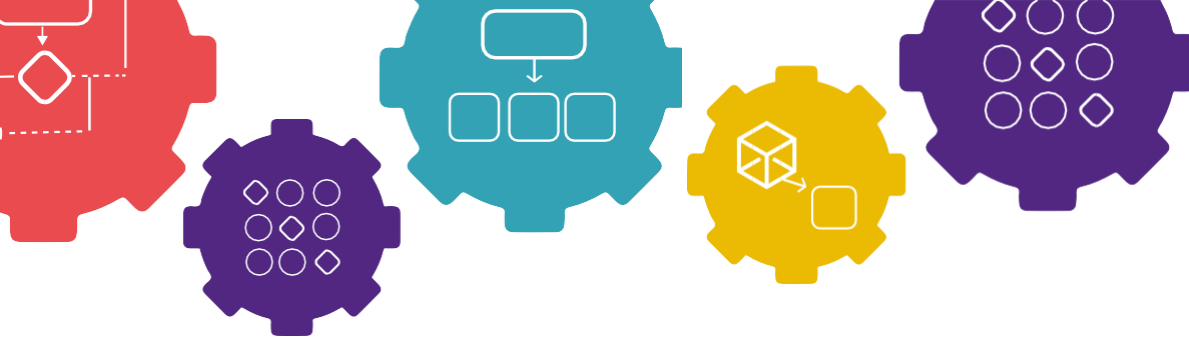


Seguimiento paso a paso: Enfoque sistemático que permite encontrar el error mientras el programa se ejecuta. Se emplea la descomposición para corregir comportamiento inesperados en tiempo real.



Ejercicios

1. **Tipos de Defectos:** Seleccione la opción que mejor describa el tipo de defecto presentado.
 - a. Un programa en C# para la recopilación de datos personales muestra el siguiente mensaje en pantalla:
“Ingresar su fecha de nacimiento y presionar Enter: ”
Como resultado, el sistema ha sido incapaz de interpretar las respuestas de los usuarios debido a que todas vienen en formatos distintos (ej. DD/MM/YYYY, MM-DD-YYYY, YYYY-MM-DD, etc.)
 - Defecto de Ambigüedad
 - Defecto de Inconsistencia
 - Defecto Lógico
 - Defecto matemático
 - b. Se ha desarrollado un programa que calcula la suma, resta producto y cociente entre dos números ingresados por el usuario. Sin embargo, el programa presenta defectos cuando el usuario ingresa 0 en ambos valores pues no existe la división entre cero.
 - Defecto de Escritura
 - Defecto matemático
 - Defecto de Ambigüedad
 - Defecto de Inconsistencia.
 - c. Se solicita desarrollar un programa que genere 20 números pares aleatorios entre 50 y 100. Deberá mostrar el listado de números en pantalla, resaltando en color cyan los números primos, y calcular las medidas de tendencia central.
 - Defecto de Ambigüedad
 - Defecto de Escritura
 - Defecto matemático
 - Defecto de Inconsistencia.



- d. Un sistema bancario en línea requiere que la contraseña de sus usuarios contenga entre 8 y 16 caracteres. A pesar de que se implementó una validación recientemente, se han detectado múltiples incidencias de fallo.

```
if (longitud >= 8 || longitud <= 16) {  
    AceptarContraseña(contraseña)  
} else{  
    RechazarContraseña(contraseña)  
}
```

- Defecto de Ambigüedad
- Defecto de Escritura
- Defecto lógico
- Defecto de Inconsistencia.

2. Pruebas de calidad (testing) – bottom up:

Elabore un plan de pruebas para validar el correcto funcionamiento de los siguientes métodos. Para cada método deberá realizar tres pruebas, indicando:

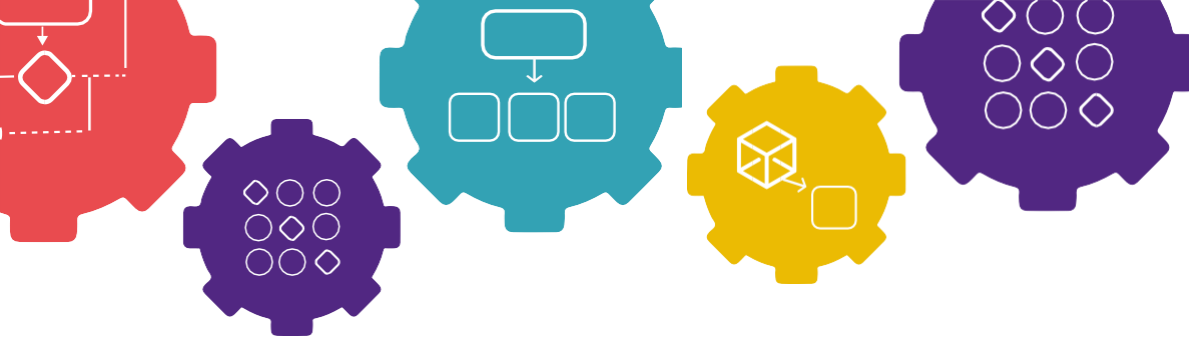
- Descripción de la prueba
- Valores de entrada
- Resultado Esperado
- Resultado obtenido, si el método cumple o no la prueba.

El objetivo del ejercicio es evidenciar, si los hay, los defectos en cada método y sugerir una propuesta de mejora.

Ejemplo: El método `CalcularPromedio( )` recibe tres notas entre 0 y 100 y retorna el promedio de las tres.

```
0 referencias  
static double CalcularPromedio(double nota1, double nota2, double nota3) {  
    return (nota1 + nota2 + nota3) / 3;  
}
```

Caso	Entrada	Resultado Esperado	Resultado Obt.
Notas dentro del rango	70, 80, 90	80	Aprobado
Nota Negativa	-10, 80, 90	Error o Rechazo	Reprobada
Nota mayor a 100	120, 80, 90	Error o Rechazo	Reprobada



Propuesta: Añadir una validación que determine si las entradas se encuentran en el rango de 0 a 100 antes de realizar los cálculos.

- a. El método `RetirarEfectivo()` permite retirar dinero de una cuenta bancaria por medio de un cajero automático. Esta recibe dos parámetros; El saldo disponible en la cuenta y el monto que desea retirar.

```
0 referencias
static void RetirarEfectivo(ref double saldo, double montoRetirado) {
    saldo = saldo - montoRetirado;
}
```

Caso	Entrada	Resultado Esperado	Resultado Obt.
Retiro valido	saldo: 500 montoRetirado: 100	Saldo = 400	Aprobado
Retiro mayor al saldo	Saldo: 500 montoRetirado:600	Saldo = -100 (saldo insuficiente)	Reprobado
Retiro negativo	Saldo: 500 montoRetirad: 200	-Error monto invalido	Reprobado

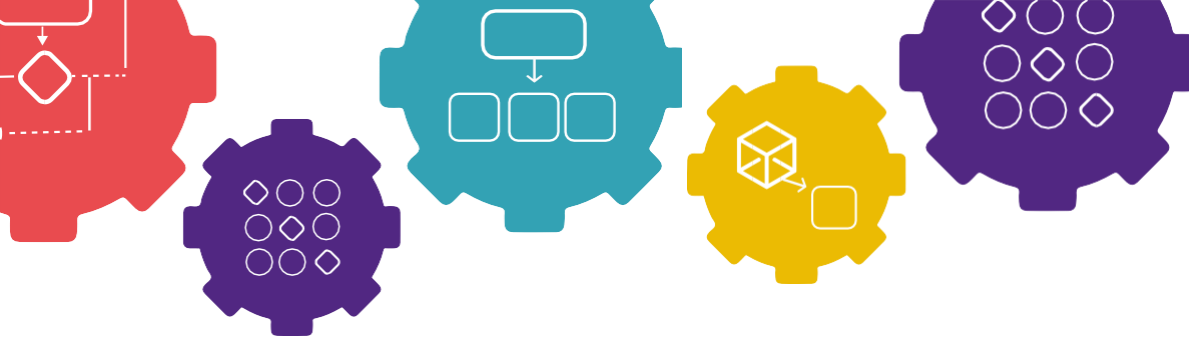
Propuesta: Se pueden agregar validaciones que verifiquen que el monto a retirar se mayor que 0 y que no sea mayor al saldo disponible antes de realizar la resta.

- b. La función `CalcularDescuento()` permite aplicar un descuento o rebaja al precio original de un producto. Para esto utiliza dos entradas; el precio original del producto y el porcentaje de descuento, entre 0 y 100, que debemos aplicar.

```
0 referencias
static double CalcularDescuento(double precio, double porcentaje)
{
    return precio - (precio * porcentaje / 100);
}
```

Caso	Entrada	Resultado Esperado	Resultado Obt.
Descuento valido	Precio: 100, porcentaje: 10	90	Aprobada
Descuento negativo	Precio: 100, porcentaje:-10	El porcentaje debe estar entre 0 y 100.	Reprobada
Descuento mayor a 100	Precio: 100, porcentaje:110	El porcentaje debe estar entre 0 y 100.	Reprobada
Precio negativo	Precio: -10 Porcentaje: 10	El precio debe ser positivo	Reprobada

Propuesta: Agregar una validación para comprobar que el precio original sea mayor que 0 y que el porcentaje de descuento esté entre 0 y 100 antes de calcular el precio final.

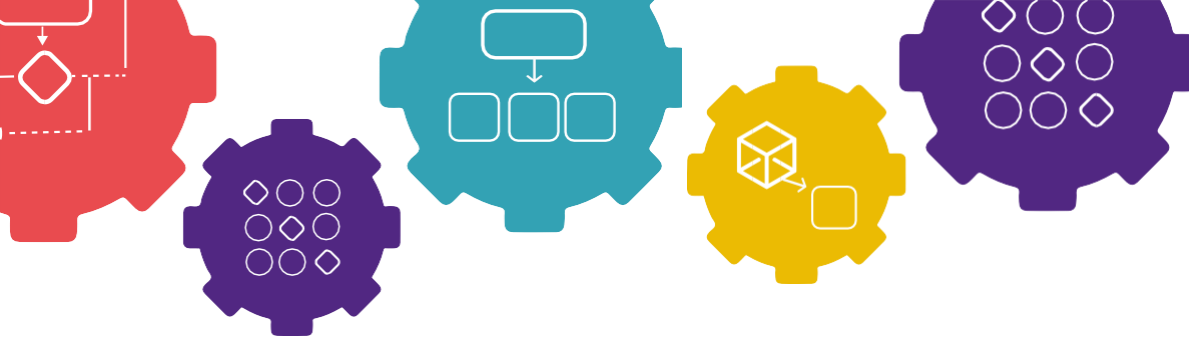


- c. El método `Depositar()` permite aumentar el saldo de una cuenta bancaria. Este recibe como entrada el saldo disponible del cliente y el monto que desea depositar.

```
0 referencias
static void Depositar(ref double saldo, double monto)
{
    saldo = saldo + monto;
}
```

Caso	Entrada	Resultado Esperado	Resultado Obt.
Deposito valido	Saldo: 500, monto: 200	700	Aprobado
Monto negativo	Saldo: 500, monto:-100	El monto debe ser positivo	Reprobado
Deposito igual a cero	Saldo: 500, monto: 0	El monto debe ser diferente de 0	Reprobado
Saldo menor a 0	Saldo: -100, monto:50	El saldo debe ser positivo	Reprobado

Propuesta: Agregar una validación para comprobar que el saldo original sea positivo y una validación para que el monto ingresado sea mayor a 0.



3. Seguimiento paso a paso (Debugging):

Crear un proyecto de consola en C# e ingresar el código proporcionado para simular el pago de un crédito personal.

```
static void Main()
{
    double capital = 1000;
    double tasa = 0.05;
    double intereses = 0;
    double abonos = 0;

    for (int mes = 1; mes <= 12 && capital > 0; mes++)
    {
        // Cálculo de intereses del mes
        intereses = capital * tasa;

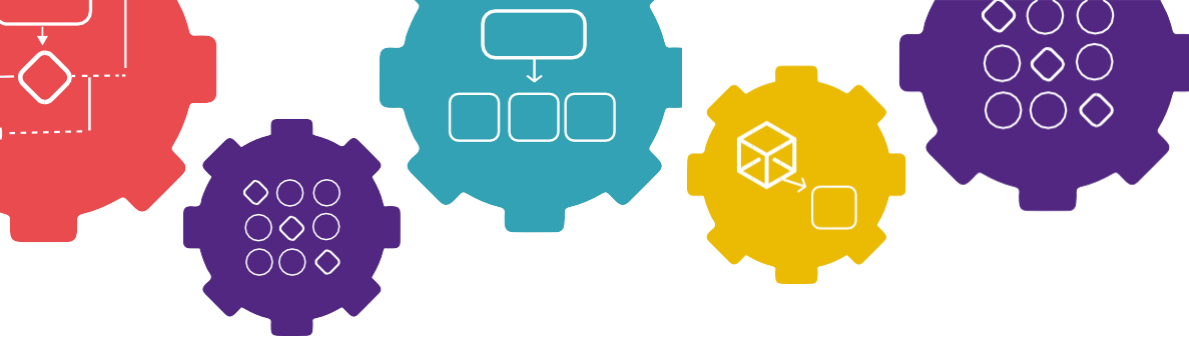
        // Abono realizado cada mes
        abonos = 100 + (mes * 10);

        // Actualización del capital
        capital = capital + intereses - abonos;
    }
}
```

Deberá utilizar herramientas de depuración (*debugging*) mediante el uso de **breakpoints** para observar cómo cambian las variables durante la ejecución del programa.

Instrucciones:

- Ejecute el programa en modo depuración (*Debug*).
- Coloque un **breakpoint** dentro del ciclo for, específicamente en la línea donde se actualiza el capital.
- Utilice las herramientas de depuración:
 - Step Over (F10)
 - Ventana de variables locales
- Observe cómo cambian las siguientes variables durante cada iteración:
 - capital
 - intereses
 - abonos



- tasa
 - mes
- e. Tome capturas de pantalla donde se visualicen claramente los valores de las variables al finalizar:
- Mes 2
 - Mes 4
 - Mes 6
 - Mes 8

Mes 2:

```

1
2
3 double capital = 1000; //se le asigna el valor
4 double tasa = 0.05;
5 double intereses = 0;
6 double abonos = 0;
7 Console.WriteLine("aaaaaaa");
8 for (int mes = 1; mes <= 12 && capital > 0; mes++) [mes:2]
9 {
10     // Cálculo de intereses del mes
11     intereses = capital * tasa;
12
13     // Abono realizado cada mes
14     abonos = 100 * (mes * 10);
15
16     // Actualización del capital
17     capital = capital + intereses - abonos;
18 }
19

```

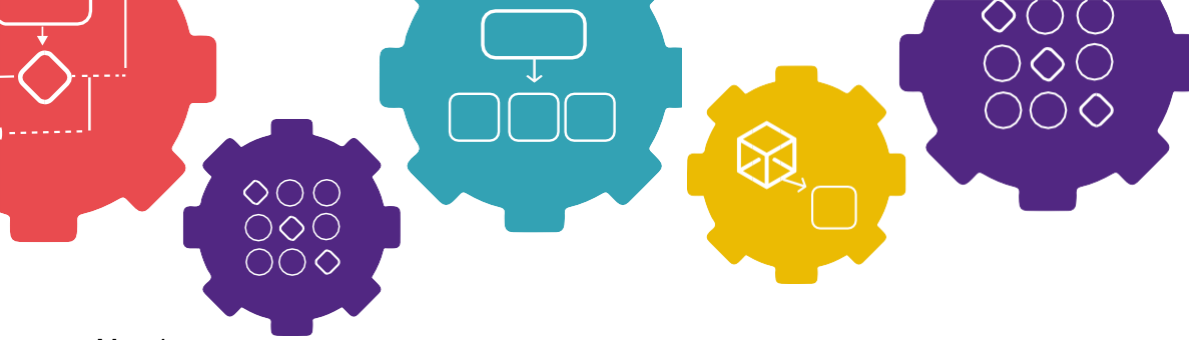
Name	Value	Type
abonos	120	double
capital	940	double
intereses	47	double
mes	2	int

```

1
2
3 double capital = 1000; //se le asigna el valor
4 double tasa = 0.05;
5 double intereses = 0;
6 double abonos = 0;
7 Console.WriteLine("aaaaaaa");
8 for (int mes = 1; mes <= 12 && capital > 0; mes++) [mes:5]
9 {
10     // Cálculo de intereses del mes
11     intereses = capital * tasa;
12
13     // Abono realizado cada mes
14     abonos = 100 * (mes * 10);
15
16     // Actualización del capital
17     capital = capital + intereses - abonos;
18 }
19

```

Name	Value	Type
abonos	120	double
capital	867	double
intereses	47	double



Mes 4:

Visual Studio Code interface showing the execution of a C# program. The code is in `L15_JCAA_1227526.cs` and is being debugged. The program calculates the interest and capital for a given month.

```
1
2
3 double capital = 1000;
4 double tasa = 0.05;
5 double intereses = 0;
6 double abonos = 0;
7 Console.WriteLine("aaaaaa");
8 for (int mes = 1; mes <= 12 && capital > 0; mes++)
9 {
10     // Cálculo de intereses del mes
11     intereses = capital * tasa;
12     // Abono realizado cada mes
13     abonos = 100 * (mes + 10);
14     // Actualización del capital
15     capital = capital + intereses - abonos;
16 }
17
18
19
```

The Autos window shows the following variables:

Name	Value	Type
abonos	140	double
capital	780.35	double
intereses	39.017500000000005	double
mes	4	int

The Error List shows 0 Errors, 0 Warnings, and 0 of 2 Messages.

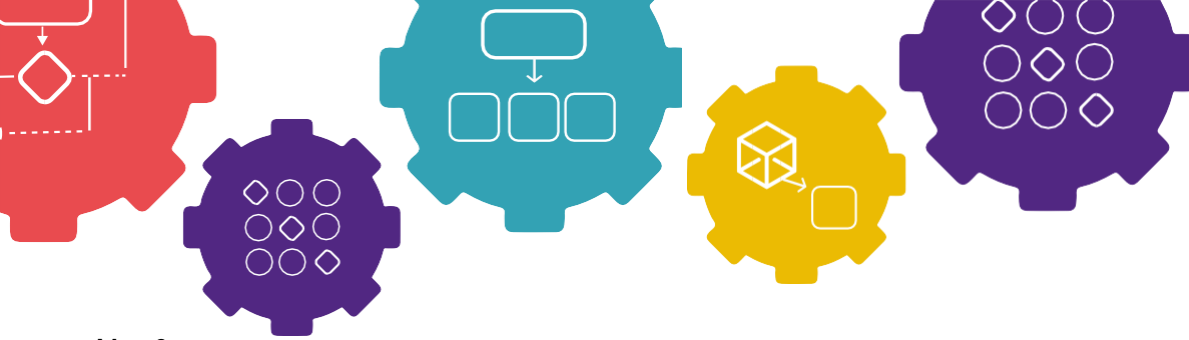
Visual Studio Code interface showing the execution of the same C# program. The code is in `L15_JCAA_1227526.cs` and is being debugged. The program calculates the interest and capital for a given month.

```
1
2
3 double capital = 1000;
4 double tasa = 0.05;
5 double intereses = 0;
6 double abonos = 0;
7 Console.WriteLine("aaaaaa");
8 for (int mes = 1; mes <= 12 && capital > 0; mes++)
9 {
10     // Cálculo de intereses del mes
11     intereses = capital * tasa;
12     // Abono realizado cada mes
13     abonos = 100 * (mes + 10);
14     // Actualización del capital
15     capital = capital + intereses - abonos;
16 }
17
18
19
```

The Autos window shows the following variables:

Name	Value	Type
abonos	140	double
capital	679.36750000000006	double
intereses	39.017500000000005	double

The Error List shows 0 Errors, 0 Warnings, and 0 of 2 Messages.



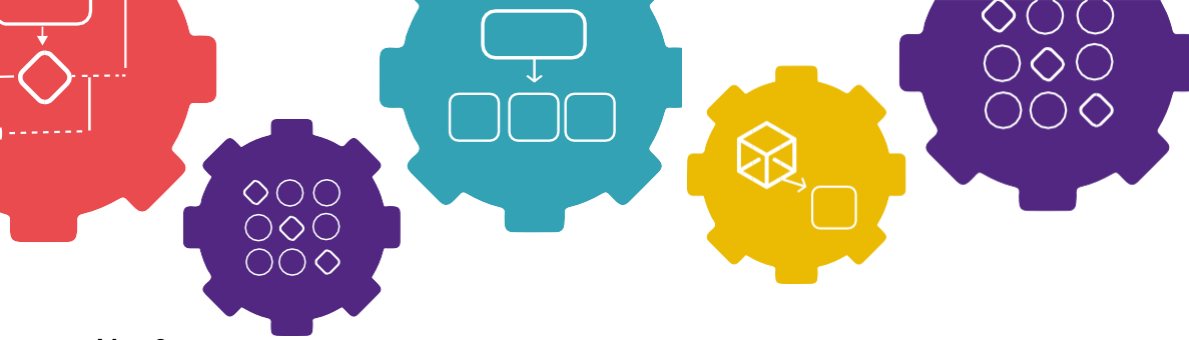
Mes 6:

```
1
2
3 double capital = 1000;
4 double tasa = 0.05;
5 double intereses = 0;
6 double abonos = 0;
7 Console.WriteLine("aaaaaaa");
8 for (int mes = 1; mes <= 12 && capital > 0; mes++)
9 {
10     // Cálculo de intereses del mes
11     intereses = capital * tasa;
12     // Abono realizado cada mes
13     abonos = 100 * (mes + 10);
14     // Actualización del capital
15     capital = capital + intereses - abonos;
16 }
17
18
19
```

Name	Value	Type
abonos	100	double
capital	563.3358750000001	double
intereses	28.166793750000007	double
mes	6	int

```
1
2
3 double capital = 1000;
4 double tasa = 0.05;
5 double intereses = 0;
6 double abonos = 0;
7 Console.WriteLine("aaaaaaa");
8 for (int mes = 1; mes <= 12 && capital > 0; mes++)
9 {
10     // Cálculo de intereses del mes
11     intereses = capital * tasa;
12     // Abono realizado cada mes
13     abonos = 100 * (mes + 10);
14     // Actualización del capital
15     capital = capital + intereses - abonos;
16 }
17
18
19
```

Name	Value	Type
abonos	160	double
capital	411.50266875000011	double
intereses	28.166793750000007	double



Mes 8:

Visual Studio Code interface showing the execution of a C# program. The code is in a file named `Program.cs` within the project `L15_JCAA_1227526`. The code calculates the monthly interest and capital for 12 months.

```
1 double capital = 1000; [argv [string]]
2 double tasa = 0.05;
3 double intereses = 0;
4 double abonos = 0;
5 Console.WriteLine("aaaaaa");
6 for (int mes = 1; mes <= 12 && capital > 0; mes++) [mes 8]
7 {
8     // Cálculo de intereses del mes
9     intereses = capital * tasa;
10    // Abono realizado cada mes
11    abonos = 100 + (mes * 10);
12    // Actualización del capital
13    capital = capital + intereses - abonos;
14 }
15
16
17
18
19
```

The output window shows the following values:

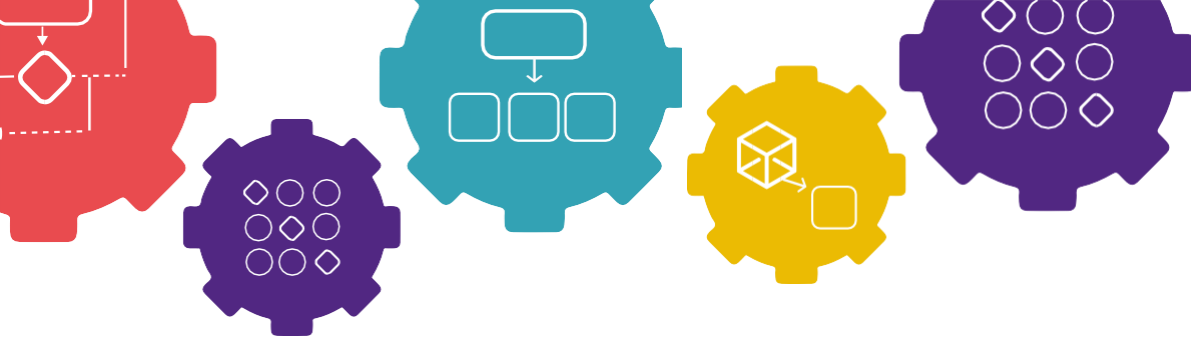
Name	Value	Type
abonos	100	double
capital	283.07780216750014	double
intereses	14.153890109375007	double
mes	8	int

Visual Studio Code interface showing the execution of the same C# program. The code is in a file named `Program.cs` within the project `L15_JCAA_1227526`. The code calculates the monthly interest and capital for 12 months.

```
1 double capital = 1000; [argv [string]]
2 double tasa = 0.05;
3 double intereses = 0;
4 double abonos = 0;
5 Console.WriteLine("aaaaaa");
6 for (int mes = 1; mes <= 12 && capital > 0; mes++) [mes 8]
7 {
8     // Cálculo de intereses del mes
9     intereses = capital * tasa;
10    // Abono realizado cada mes
11    abonos = 100 + (mes * 10);
12    // Actualización del capital
13    capital = capital + intereses - abonos;
14 }
15
16
17
18
19
```

The output window shows the following values:

Name	Value	Type
abonos	100	double
capital	117.23169229687517	double
intereses	14.153890109375007	double



Ejercicio	Criterio	Puntos	Total
Ejercicio 1: Tipos de Defectos	Inciso A	5 pts	20 pts
	Inciso B	5 pts	
	Inciso C	5 pts	
	Inciso D	5 pts	
Ejercicio 2: Testing	Diseño de pruebas adecuados, no faltan escenarios.	10 pts	40 pts
	Identifica correctamente todos los defectos presentes en las funciones	10 pts	
	Documenta de forma clara los resultados esperados y el comportamiento observado en cada prueba.	10 pts	
	Propone validaciones o correcciones adecuadas para solucionar los defectos encontrados	10 pts	
Ejercicio 3: Debugging	Estado de las variables en el Mes 2	10 pts	40 pts
	Estado de las variables en el Mes 4	10 pts	
	Estado de las variables en el Mes 6	10 pts	
	Estado de las variables en el Mes 8	10 pts	
Total			100 pts